torch.linalg.solve#

https://pytorch.org/docs/stable/generated/torch.linalg.solve.html

Description:

Computes the solution of a square system of linear equations with a unique solution. Letting K\mathbb{K}K be R\mathbb{R}R or C\mathbb{C}C, this function computes the solution X∈Kn×kX \in \mathbb{K}^{n \times k}X∈Kn×k of the linear system associated to A∈Kn×n,B∈Kn×kA \in \mathbb{K}^{n \times n}, B \in \mathbb{K}^{n \times k}A∈Kn×n,B∈Kn×k, which is defined as If left= False, this function returns the matrix X∈Kn×kX \in \mathbb{K}^{n \times k}X∈Kn×k that solves the system This system of linear equations has one solution if and only if AAA is invertible. This function assumes that AAA is invertible. Supports inputs of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if the inputs are batches of matrices then the output has the same batch dimensions.

Function Signature

```
torch.linalg.solve(A, B, *, left=True, out=None) → Tensor#
Parameters
Keyword Arguments
Raises
```

Parameters

Keyword

left (bool, optional) – whether to solve the system AX=BAX=BAX=B or XA=BXA = BXA=B. Default: True. out (Tensor, optional) – output tensor. Ignored if None. Default: None.

Code Examples

```
>>> A = torch.randn(3, 3)
>>> b = torch.randn(3)
>>> x = torch.linalg.solve(A, b)
>>> torch.allclose(A @ x, b)
True
>>> A = torch.randn(2, 3, 3)
>>> B = torch.randn(2, 3, 4)
>>> X = torch.linalg.solve(A, B)
>>> X.shape
torch.Size([2, 3, 4])
>>> torch.allclose(A @ X, B)
True

>>> A = torch.randn(2, 3, 3)
>>> b = torch.randn(3, 1)
>>> x = torch.linalg.solve(A, b) # b is broadcasted to size (2,
3, 1)
>>> x.shape
torch.Size([2, 3, 1])
>>> torch.allclose(A @ x, b)
True
>>> b = torch.randn(3)
>>> x = torch.linalg.solve(A, b) # b is broadcasted to size (2,
3)
>>> x.shape
torch.Size([2, 3])
>>> Ax = A @ x.unsqueeze(-1)
>>> torch.allclose(Ax, b.unsqueeze(-1).expand_as(Ax))
True
```

Notes & Warnings

NOTE Note This function computes X = A.inverse() @ B in a faster and more numerically stable way than performing the computations separately.

NOTE Note It is possible to compute the solution of the system XA=BXA = BXA=B by passing the inputs A and B transposed and transposing the output returned by this function.

NOTE Note A is allowed to be a non-batched torch.sparse_csr_tensor, but only with left=True.

NOTE Note When inputs are on a CUDA device, this function synchronizes that device with the CPU. For a version of this function that does not synchronize, see torch.linalg.solve_ex().

NOTE See also torch.linalg.solve_triangular() computes the solution of a triangular system of linear equations with a unique solution.

Detailed Documentation

Documentation

Computes the solution of a square system of linear equations with a unique solution.

Letting K\mathbb{K}K be R\mathbb{R}R or C\mathbb{C}C, this function computes the solution X∈Kn×kX \in \mathbb{K}^{n \times k}X∈Kn×k of the linear system associated to A∈Kn×n,B∈Kn×kA \in \mathbb{K}^{n \times n}, B \in \mathbb{K}^{n \times k}A∈Kn×n,B∈Kn×k, which is defined as

If left= False, this function returns the matrix X∈Kn×kX \in \mathbb{K}^{n \times k}X∈Kn×k that solves the system

This system of linear equations has one solution if and only if AAA is invertible. This function assumes that AAA is invertible.

Supports inputs of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if the inputs are batches of matrices then the output has the same batch dimensions.

Letting * be zero or more batch dimensions,

If A has shape (*, n, n) and B has shape (*, n) (a batch of vectors) or shape (*, n, k) (a batch of matrices or “multiple right-hand sides”), this function returns X of shape (*, n) or (*, n, k) respectively.

Otherwise, if A has shape (*, n, n) and B has shape (n,) or (n, k), B is broadcasted to have shape (*, n) or (*, n, k) respectively. This function then returns the solution of the resulting batch of systems of linear equations.

This function computes X = A.inverse() @ B in a faster and more numerically stable way than performing the computations separately.

It is possible to compute the solution of the system XA=BXA = BXA=B by passing the inputs A and B transposed and transposing the output returned by this function.

A is allowed to be a non-batched torch.sparse_csr_tensor, but only with left=True.

When inputs are on a CUDA device, this function synchronizes that device with the CPU. For a version of this function that does not synchronize, see torch.linalg.solve_ex().

torch.linalg.solve_triangular() computes the solution of a triangular system of linear equations with a unique solution.

A (Tensor) – tensor of shape (*, n, n) where * is zero or more batch dimensions.

B (Tensor) – right-hand side tensor of shape (*, n) or (*, n, k) or (n,) or (n, k) according to the rules described above

left (bool, optional) – whether to solve the system AX=BAX=BAX=B or XA=BXA = BXA=B. Default: True.

out (Tensor, optional) – output tensor. Ignored if None. Default: None.

RuntimeError – if the A matrix is not invertible or any matrix in a batched A is not invertible.